Data Mining And Warehousing

B.Tech CSE (3rd Year) (Dr. A.P.J. Abdul Kalam Technical University)

Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

Data Warehousing and Data Mining (BCS058)

Unit 1: Overview, Definition, Data Warehousing Components, Building a Data Warehouse, Warehouse Database, Mapping the Data Warehouse to a Multiprocessor Architecture, Difference between Database System and Data Warehouse, Multi Dimensional Data Model, Data Cubes, Stars, Snow Flakes, Fact Constellations, Concept

Unit 2: Data Warehouse Process and Technology: Warehousing Strategy, Warehouse /management and Support Processes, Warehouse Planning and Implementation, Hardware and Operating Systems for Data Warehousing, Client/Server Computing Model & Data Warehousing. Parallel Processors & Cluster Systems, Distributed DBMS implementations, Warehousing Software, Warehouse Schema Design,

Unit 3: Data Mining: Overview, Motivation, Definition & Functionalities, Data Processing, Form of Data Pre-processing, Data Cleaning: Missing Values, Noisy Data, (Binning, Clustering, Regression, Computer and Human inspection), Inconsistent Data, Data Integration and Transformation. Data Reduction:-Data Cube Aggregation, Dimensionality reduction, Data Compression, Numerosity Reduction, Discretization and Concept hierarchy generation, Decision Tree.

Unit 4: Classification: Definition, Data Generalization, Analytical Characterization, Analysis of attribute relevance, Mining Class comparisons, Statistical measures in large Databases, Statistical-Based Algorithms, Distance-Based Algorithms, Decision Tree-Based Algorithms. Clustering: Introduction, Similarity and Distance Measures, Hierarchical and Partitional Algorithms. Hierarchical ClusteringCURE and Chameleon. Density Based Methods-DBSCAN, OPTICS. Grid Based Methods- STING, CLIQUE. Model Based Method –Statistical Approach, Association rules: Introduction, Large Item sets, Basic Algorithms, Parallel and Distributed Algorithms, Neural Network approach.

Unit 5: Data Visualization and Overall Perspective: Aggregation, Historical information, Query Facility, OLAP function and Tools. OLAP Servers, ROLAP, MOLAP, HOLAP, Data Mining interface, Security, Backup and Recovery, Tuning Data Warehouse, Testing Data Warehouse. Warehousing applications and Recent Trends: Types of Warehousing Applications, Web Mining, Spatial Mining and Temporal Mining

This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

Unit 1

Definition: Data warehousing refers to the process of collecting, storing, and managing large amounts of structured and sometimes semi-structured data from multiple sources in a centralized repository. This repository, known as a data warehouse, is designed to facilitate efficient querying, analysis, and reporting of data. It supports decision-making by allowing organizations to store historical data and perform complex queries on it to generate insights, trends, and reports**.**

Data warehousing component:

1. Data Sources

- These are the various systems and databases from which the data is extracted. They can be:
- Operational databases (e.g., relational databases, transactional systems)
-
- External data sources (e.g., third-party data providers)
- Legacy systems

2. ETL Process (Extract, Transform, Load)

- **Extract:** Data is extracted from various data sources.
- **Transform:** The extracted data is cleaned, validated, and transformed into a consistent format suitable for analysis.
- **Load:** The transformed data is then loaded into the data warehouse.

This process ensures that the data is integrated and stored in a format that supports querying and analysis.

3. Data Storage (Data Warehouse Repository)

This is the central repository where the processed data is stored. It is designed to handle large volumes of data and provide fast querying capabilities.

- **Fact tables:** Store quantitative data for analysis (e.g., sales, revenue).
- **Dimension tables:** Store descriptive information related to facts (e.g., customer, product).

4. Metadata

- Metadata is "data about data." It provides information about the data stored in the warehouse, such as data definitions, mappings, source system details, transformation rules, and relationships between tables. It helps users understand the data structure and lineage for effective analysis.

5. Data Marts

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

- Data marts are smaller, specialized subsets of the data warehouse, designed to serve the needs of specific business units or departments (e.g., marketing, finance). They allow for focused, quick access to data relevant to specific areas without querying the entire data warehouse.

6. OLAP (Online Analytical Processing)

- OLAP tools are used to perform multidimensional analysis of the data stored in the warehouse. They allow users to slice, dice, drill down, and pivot data to explore different perspectives and insights.

7. Data Warehouse Access Tools

- **Query tools:** Allow users to run ad-hoc queries and retrieve data.
- **Reporting tools:** Generate reports based on predefined templates and criteria.
- **Business Intelligence (BI) tools:** Provide advanced analysis, visualizations, and dashboard

Building a Data Warehouse:

Building a data warehouse involves designing a system that aggregates, organizes, and stores data from various sources for efficient querying, reporting, and analysis. Here are the main steps to building a data warehouse:

1. Requirements Gathering

- **Business Objectives:** Understand the key business needs and goals, such as reporting, analysis, or decision-making.
- **Data Sources:** Identify the source systems (databases, APIs, flat files, etc.) that will provide the raw data for the warehouse.
- **User Requirements:** Determine what data the end-users need, how frequently they need it, and in what format (e.g., dashboards, reports).

2. Data Mode-ling

- **Conceptual Design:** Define the high-level structure of the data warehouse. This usually involves creating an entity-relationship (ER) diagram to show the data flow.
- **Dimensional Mode-ling**: Design the warehouse using star or snowflake schemas. These schemas consist of:
- **Fact Tables:** Store quantitative data (e.g., sales, revenue).
- **Dimension Tables:** Store descriptive information (e.g., product, time, geography) that provides context to the facts.
- **Normalization vs. Denormalization**: While source systems might be normalized for efficiency, the warehouse design typically favors denormalization for faster querying.

This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

3. ETL Process (Extract, Transform, Load)

- **Extract:** Pull data from the source systems. This can be done in batch (e.g., once a day) or in real-time (e.g., streaming).
- **Transform:** Clean and standardize the data. This includes:

Removing duplicates

Handling missing values

Conforming data formats

Aggregating or calculating derived metrics

- **Load:** Insert the transformed data into the warehouse. This is often done incrementally to handle new or changed data without duplicating existing data.

4. Database Selection and Infrastructure

- **Choose a Database System:** Common options include:

Relational Databases (e.g., PostgreSQL, MySQL)

Columnar Databases (e.g., Amazon Redshift, Google Big Query, Snowflake)

Data Lakes (e.g., Amazon S3, Azure Data Lake)

- **On-premise vs. Cloud**: Decide whether to host the warehouse on-premise or in the cloud. Cloud platforms like AWS, Azure, or Google Cloud offer scalability, ease of management, and integration with many services.

5. Data Integration and Governance

- **Data Integration:** Ensure that data from disparate sources is integrated meaningfully. This includes handling different formats, timestamps, and ensuring consistent identifiers (e.g., customer ID).
- **Data Quality:** Implement data validation, error logging, and monitoring tools to ensure data integrity.
- **Data Governance:** Establish rules and procedures to manage the lifecycle of the data, including access control, data lineage, privacy policies (e.g., GDPR compliance).

Warehouse Database:

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

A Warehouse Database is a structured collection of data used to manage and track various operations within a warehouse. It supports functions like inventory management, order processing, shipping, receiving, and overall logistics. It's commonly part of a Warehouse Management System (WMS).

Key Components of a Warehouse Database:

1.Inventory Management:

- Tracks the stock levels of products in the warehouse.
- Monitors product quantities, stock location, and inventory movement (inbound/outbound).

2.Order Processing:

- Handles customer orders, from receiving orders to shipping the products.
- Stores details about order status, fulfilment times, and any backorders.

3.Shipping and Receiving:

- Manages incoming shipments (goods coming into the warehouse) and outgoing shipments (dispatches to customers).
- Keeps track of delivery dates, carrier information, and shipment status.
- Supports coordination between warehouse, suppliers, and customers.

4.Locations and Bins:

- Stores information on the layout of the warehouse, including aisles, shelves, racks, and bins.
- Optimizes picking paths for warehouse staff to reduce retrieval time and enhance efficiency.

5.Product Details:

- Includes information like product name, SKU (Stock Keeping Unit), dimensions, weight, and category.
- May include additional attributes like expiry dates, lot numbers, or batch tracking, particularly for industries like food or pharmaceuticals.

6.Employee and Role Management:

- Contains employee details and their roles in the warehouse.
- Manages permissions for specific tasks (e.g., certain employees can only access inventory, while others can handle shipping).

7.Supplier Information:

This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

- Tracks data on vendors and suppliers.
- Stores information such as lead times, supplier reliability, and payment terms.

Common Tables in a Warehouse Database:

- **Products:** Contains product details like SKU, description, category, and unit price.
- **Inventory:** Tracks stock levels, product locations, lot numbers, and inventory status (available, reserved, in transit).
- **Orders:** Includes details on customer orders, order status (pending, fulfilled, shipped), and associated products.
- **Suppliers:** Stores data on suppliers, purchase orders, and delivery schedules.
- **Shipping:** Holds information about shipments, delivery addresses, and carrier tracking.
- **Employees:** Manages employee roles, tasks, and login credentials for the WMS.

Mapping the Data Warehouse to a Multiprocessor Architecture:

1. Relational data base technology for data warehouse

- **Linear Speed up**: refers the ability to increase the number of processors to reduce

response time

- **Linear Scale up:** refers the ability to provide same performance on the same requests as

the database size increases

1.1 Types of parallelism

I. Inter query Parallelism:

In which different server threads or processes handle multiple requests at the sametime**.**

II. Intra query Parallelism:

- This form of parallelism decomposes the serial SQL query into lower-level operations
- such as scan, join, sort etc. Then these lower-level operations are executed concurrentlyin parallel.

- Intra query parallelism can be done in either of two ways:

- **Horizontal parallelism:** which means that the data base is partitioned across multipledisks and parallel processing occur within a specific task that is performed concurrently on different processors against different set of data
- **Vertical parallelism**: This occurs among different tasks. All query components such as
- scan, join, sort etc are executed in parallel in a pipelined fashion. In other words, anoutput from one task becomes an input into another task.

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

III. Data partitioning:

Data partitioning is the key component for effective parallel execution of data baseoperations. Partition can be done randomly or intelligently.

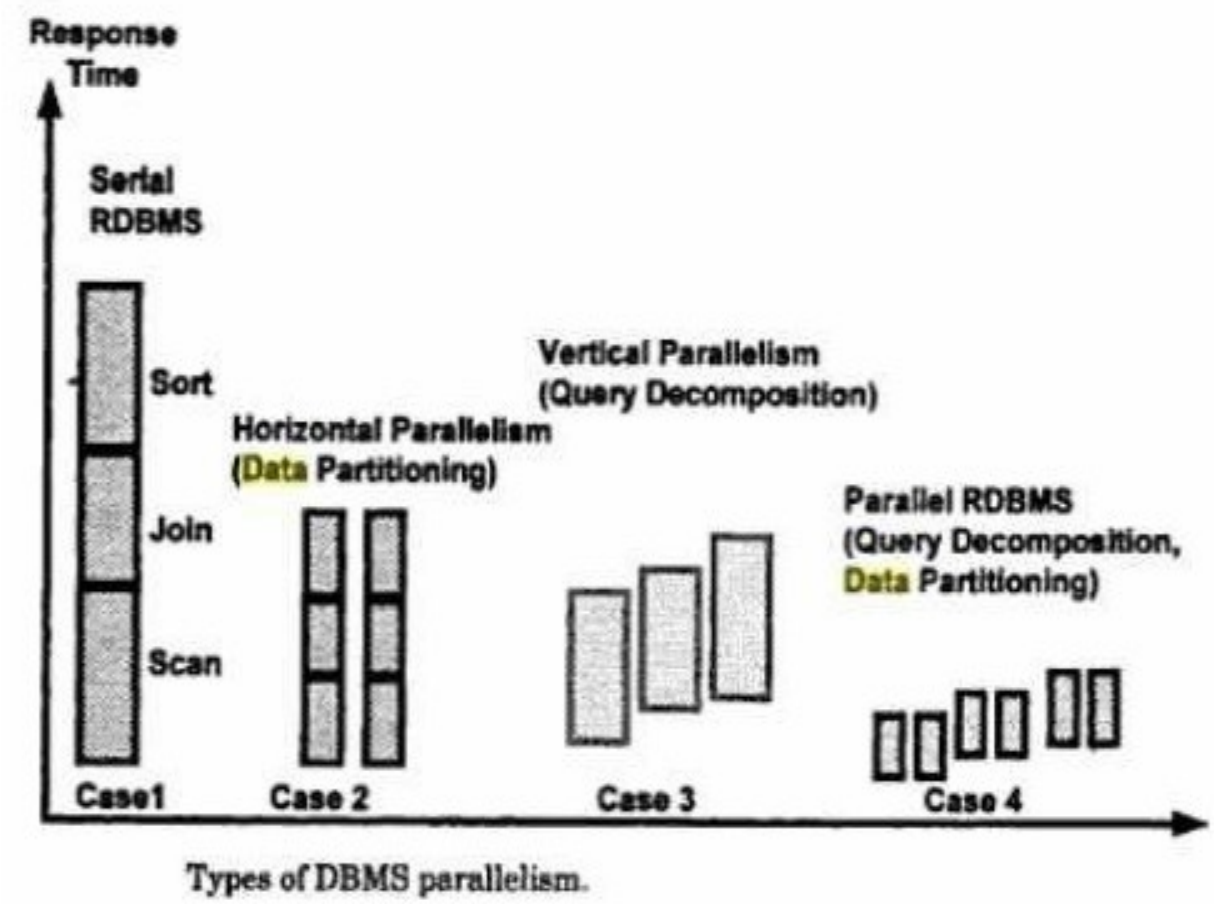
- **Random portioning:** Includes random data striping across multiple disks on asingle server.
- **Intelligent partitioning:** Assumes that DBMS knows where a specific record is
- located and does not waste time searching for it across all disks. The various

intelligent partitioning include:

- **Hash partitioning:** A hash algorithm is used to calculate the partition numberbased on the value of the partitioning key for each row
- **Key range partitioning:** Rows are placed and located in the partitions accordingto the value of the partitioning key. That is all the rows with the key value from Ato K are in partition 1, L to T is in partition 2 and so on.
- **Schema portioning:** an entire table is placed on one disk; another table is placedon different disk etc. This is useful for small reference tables.
- **User defined portioning:** It allows a table to be partitioned on the basis of a userdefined expression.

This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)



Types of DBMS parallelism.

2. Data base architectures of parallel processing

There are three DBMS software architecture styles for parallel processing:

Shared memory or shared everything Architecture
Shared disk architecture
Shred nothing architecture

2.1 Shared Memory (or Shared Everything) Architecture:

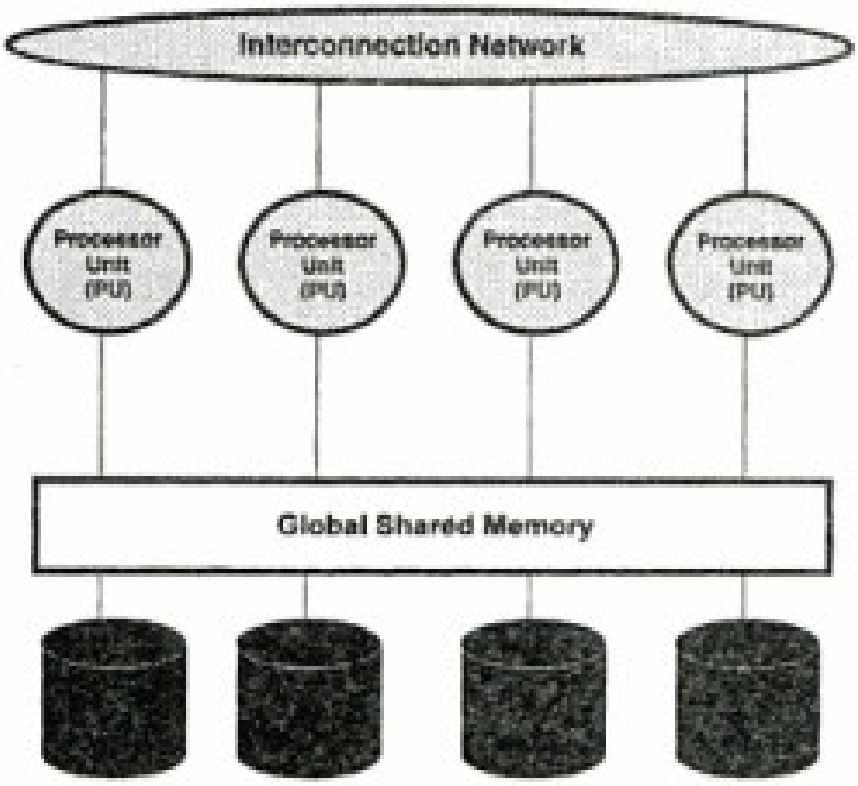
It has the following characteristics-

Multiple PUs share memory.
Each PU has full access to all shared memory through a common bus.
Communication between nodes occurs via shared memory.
Performance is limited by the bandwidth of the memory bus.

Disadvantage

Scalability is limited by bus bandwidth and latency, and by available memory.

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)



Shared-memory architecture.

2.2 Shared Disk Architecture

Shared disk systems are typically loosely coupled. Such systems, illustrated in figure, have the following characteristics:

Each node consists of one or more PUs and associated memory.
Memory is not shared between nodes.
Communication occurs over a common high-speed bus.
Each node has access to the same disks and other resources.
Bandwidth of the high-speed bus limits the number of nodes (scalability) of thesystem.

Advantages

Shared disk systems permit high availability. All data is accessible even if onenode dies.
These systems have the concept of one database, which is an advantage overshared nothing systems.
Shared disk systems provide for incremental growth.

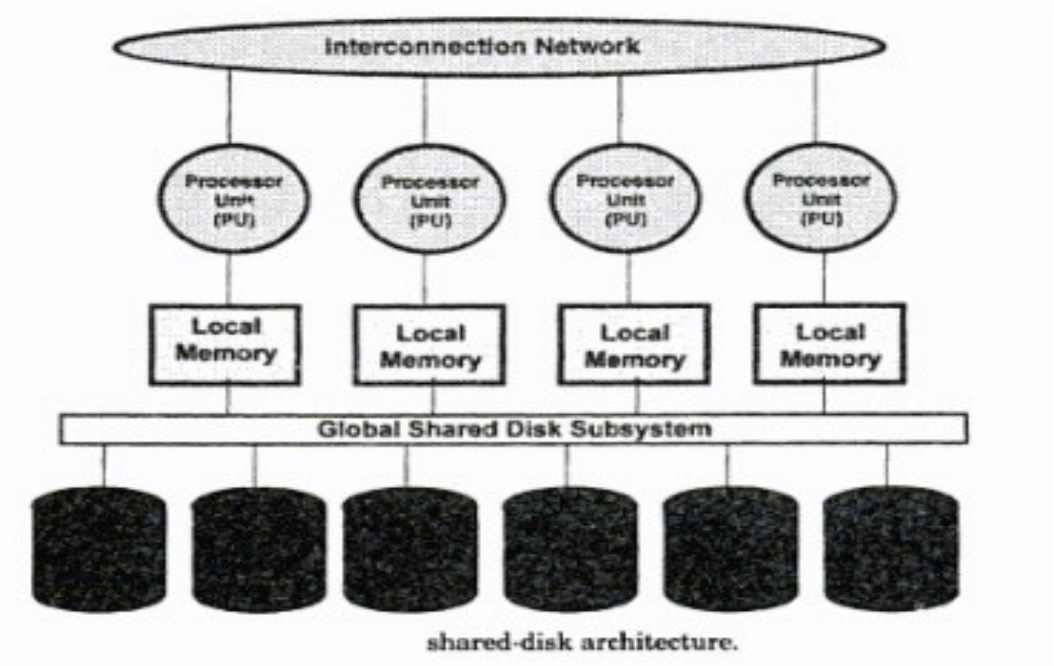
This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

Disadvantages

Inter-node synchronization is required, involving DLM overhead and greater dependency on high-speed interconnect.

If the workload is not partitioned well, there may be high synchronizationoverhead.



shared-disk architecture.

2.2 Shared Nothing Architecture (Distributed)

Shared nothing systems are typically loosely coupled. In shared nothing systems onlyone CPU is connected to a given disk. If a table or database is located on that diskShared nothing systems are concerned with access to disks, not access to memory.

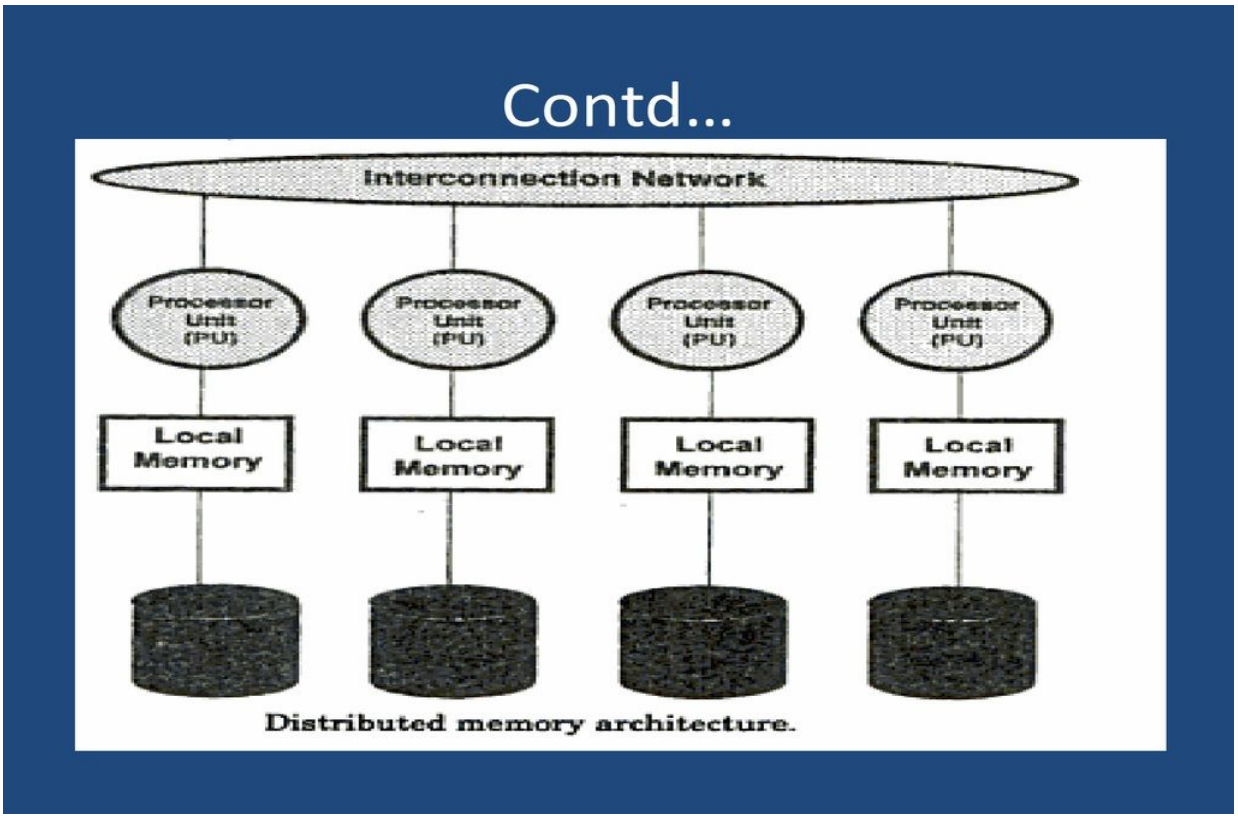
Advantages for parallel processing

Shared nothing systems provide for incremental growth.
System growth is practically unlimited.
MPPs are good for read-only databases and decision support applications.
Failure is local: if one node fails, the others stay up.

Disadvantages for parallel processing

More coordination is required.
More overhead is required for a process working on a disk belonging to anothernode.

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)



Multidimensional Data Model

A Multidimensional Data Model is a key concept in data warehousing and OLAP (Online Analytical Processing), designed to organize data for efficient querying and reporting. It represents data as a cube structure, where each dimension corresponds to a different perspective or attribute of the data, allowing complex queries to be broken down into simpler ones.

Here are the main components of a multidimensional data model:

1. Facts:

Fact tables contain the numerical data (metrics) that businesses are interested in, such as sales revenue, quantity sold, or profit.

Fact tables are typically made up of:

Measures: The quantitative values (e.g., sales, profit, revenue).

Foreign keys: Links to the dimension tables that provide context for the facts.

2. Dimensions

Dimension tables store descriptive information that defines the context for facts (e.g., time, product, geography).

Each dimension table contains attributes that provide more detail. For example, a product dimension could have attributes like product name, category, and brand.

This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

3. Schema Types

Star Schema:

The simplest form of the multidimensional model.
Consists of a central fact table linked to dimension tables.
The fact table has foreign keys connecting to dimension tables.

Snowflake Schema:

A more normalized version of the star schema.
Dimension tables are broken down into smaller tables to reduce data redundancy.
It is more complex but can save storage space by normalizing dimensions.

Galaxy Schema (Fact Constellation):

A combination of multiple star schemas.
Involves more than one fact table sharing dimension tables, useful for handling complex systems.

4. OLAP Operations

Multidimensional models allow for a variety of operations to explore the data cube:

Slice: Selects a specific dimension to filter the cube (e.g., sales in 2023).

Dice: Selects a subset of data by specifying ranges across dimensions (e.g., sales in 2023 for Product A in Region X).

Drill Down: Provides a more detailed view (e.g., from quarterly to monthly sales).

Roll Up: Aggregates data (e.g., from monthly to yearly sales).

Pivot (Rotate): Rotates the data cube for a different perspective.

Example of a Multidimensional Model:

Consider a sales data warehouse with:

A Fact Table that tracks sales, with metrics like sales_amount and units_sold.
Dimensions like Time, Product, and Store:
Time dimension might have attributes like year, quarter, month, and day.
Product dimension might have product_name, category, and brand.
Store dimension might have store_name, city, and region**.**

Data Cube:

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

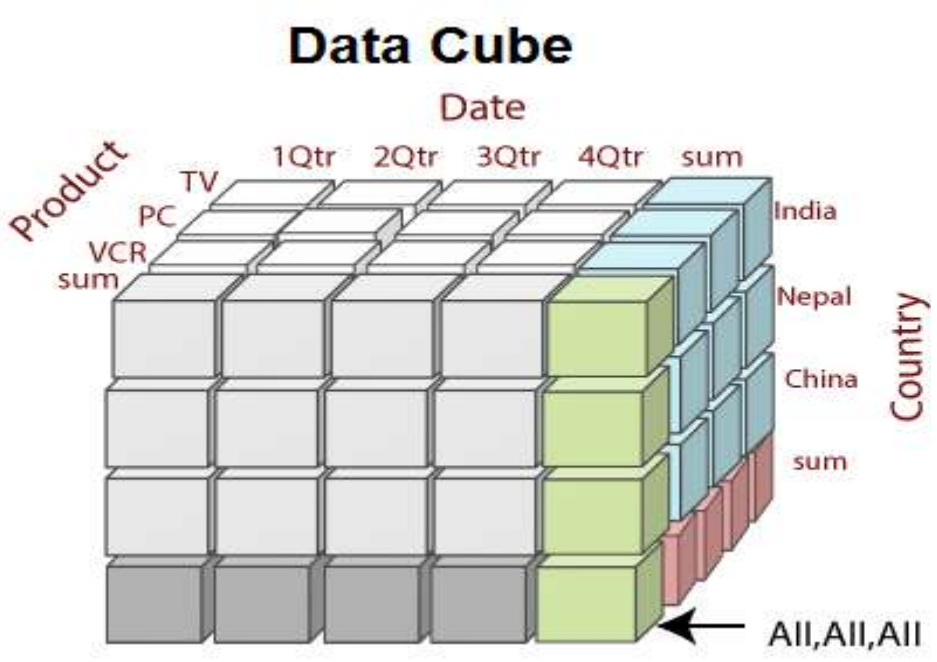
A data cube is used to represent data along various dimensions. For example, if you're analysing sales data, the dimensions could be time, region, and product category. Each cell in the cube contains aggregated data, such as total sales, for a combination of those dimension values.

Dimensions: These are perspectives or entities with respect to which an organization wants to keep records. Examples are time, geography, and product.

Facts: These are numerical measures or quantities, like sales, profit, or revenue.
Cells: Each cell in the data cube represents the aggregation (sum, average, etc.) of the data at the intersection of dimensions.
For instance, a 3D data cube could represent:
TimeRegionProductSales (fact)

Q1 2024NorthProduct A	$5000
Q1 2024SouthProduct B	$7000

Operations on Data Cubes:
Roll-up: Aggregating data by climbing up the hierarchy (e.g., from city to region).
Drill-down: Breaking data down into finer levels of detail (e.g., from year to quarter to month).
Slice: Selecting a single dimension of the cube, producing asub-cube.
Dice: Selecting two or more dimensions, creating a smaller cube.



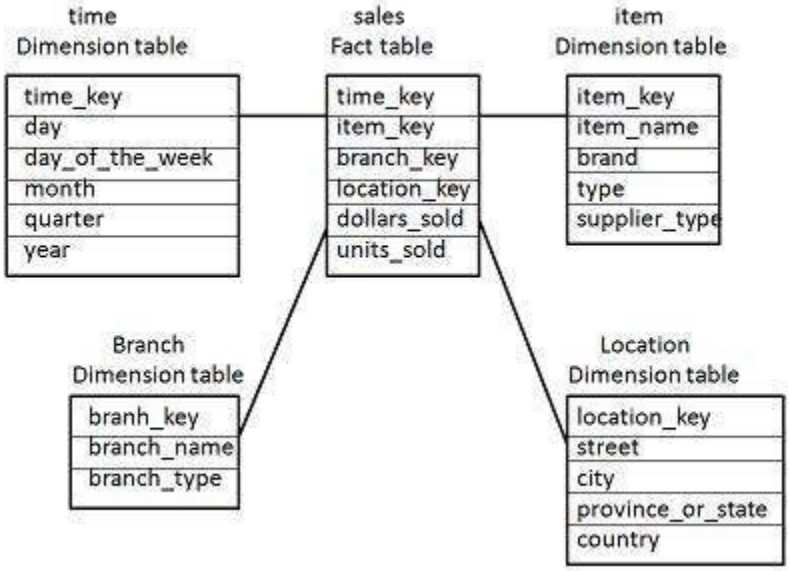
This document is available on studocu
Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

Schema Design:

Schema is a logical description of the entire database. It includes the name and description of records of all record types including all associated data-items and aggregates. Much like a database, a data warehouse also requires to maintain a schema. A database uses relational model, while a data warehouse uses Star, Snowflake, and Fact Constellation schema. In this chapter, we will discuss the schemas used in a data warehouse.

Star Schema

- Each dimension in a star schema is represented with only one-dimension table.
- This dimension table contains the set of attributes.
- The following diagram shows the sales data of a company with respect to the four dimensions, namely time, item, branch, and location.

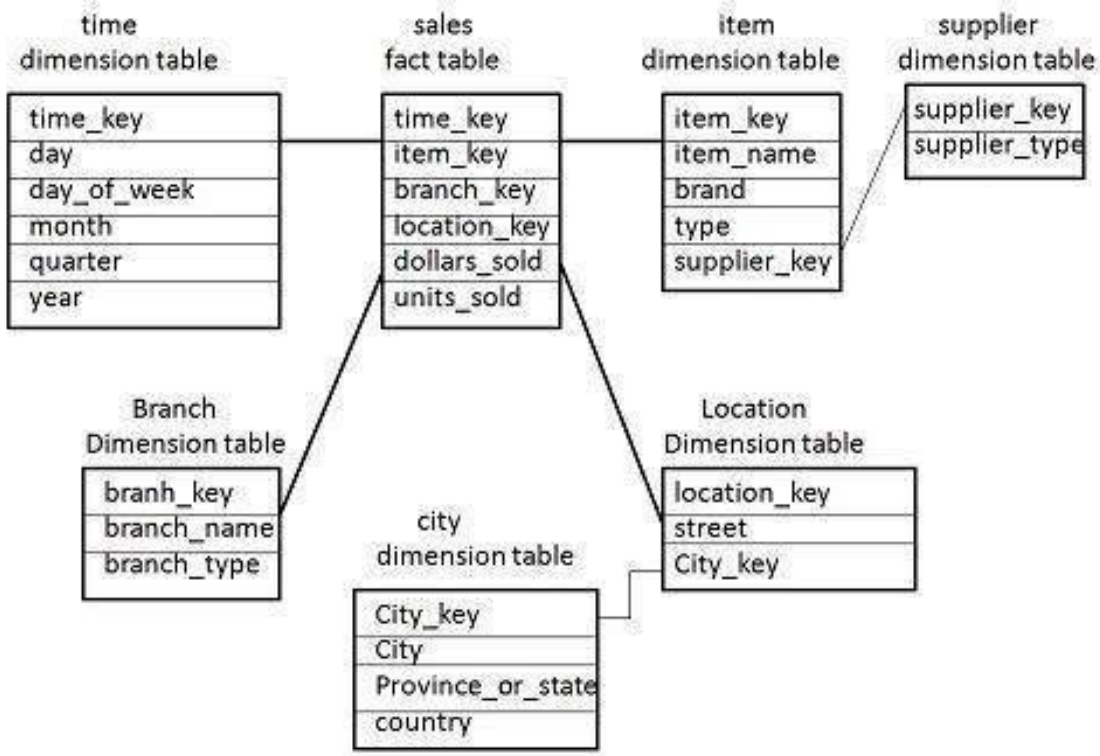


- There is a fact table at the center. It contains the keys to each of four dimensions.
- The fact table also contains the attributes, namely dollars sold and units sold.

  Note − Each dimension has only one dimension table and each table holds a set of attributes. For example, the location dimension table contains the attribute set {location_key, street, city, province_or_state,country}. This constraint may cause data redundancy. For example, "Vancouver" and "Victoria" both the cities are in the Canadian province of British Columbia. The entries for such cities may cause data redundancy along the attributesprovince_or_state and country.

  Snowflake Schema
- Some dimension tables in the Snowflake schema are normalized.
- The normalization splits up the data into additional tables.
- Unlike Star schema, the dimensions table in a snowflake schema are normalized. For example, the item dimension table in star schema is normalized and split into two dimension tables, namely item and supplier table.

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)



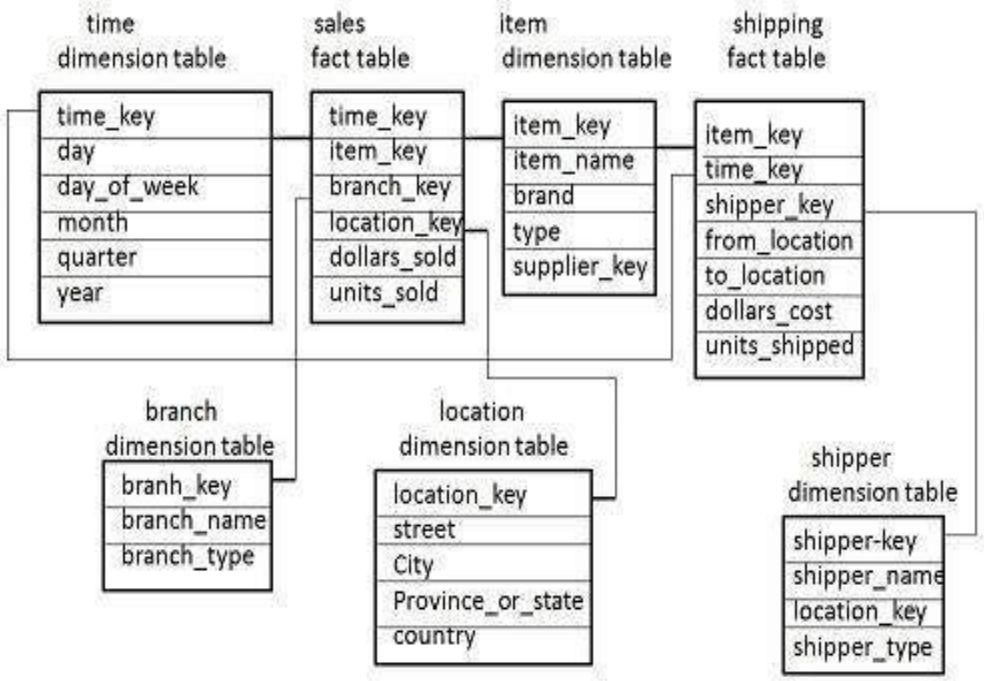
- Now the item dimension table contains the attributes item_key, item_name, type, brand, and supplier-key.
- The supplier key is linked to the supplier dimension table. The supplier dimension table contains the attributes supplier_key and supplier_type.
  Note − Due to normalization in the Snowflake schema, the redundancy is reduced and therefore, it becomes easy to maintain and the save storage space.

Fact Constellation Schema:

- A fact constellation has multiple fact tables. It is also known as galaxy schema.
- The following diagram shows two fact tables, namely sales and shipping.

This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)



- The sales fact table is same as that in the star schema.
- The shipping fact table has the five dimensions, namely item_key, time_key, shipper_key, from_location, to_location.
- The shipping fact table also contains two measures, namely dollars sold and units sold.
- It is also possible to share dimension tables between fact tables. For example, time, item, and location dimension tables are shared between the sales and shipping fact table.

Concept:
Schema design is a crucial aspect of database and data warehouse systems, especially in the context of organizing data for storage, retrieval, and efficient querying. Here are some fundamental concepts in schema design:

1. Entities and Attributes:
Entity: An entity represents a real-world object or concept (e.g., a customer, product, or order). In relational databases, entities are often mapped to tables.
Attributes: Attributes represent the properties or characteristics of an entity. For example, for a customer entity, attributes could be customer ID, name, and contact information.

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

2. Primary Key (PK):
A primary key is a unique identifier for each record in a table. It ensures that each row in a table is uniquely identifiable (e.g., a customer ID or order ID).
3. Foreign Key (FK):
A foreign key establishes a relationship between two tables. It refers to the primary key in another table, creating a link between records (e.g., linking an order to a customer by storing the customer ID in the order table).
4. Normalization:
Normalization is the process of organizing data in such a way that redundancy is minimized. It involves dividing large tables into smaller, related tables. The aim is to avoid anomalies in data operations (insert, update, delete).
1NF: Eliminate duplicate columns and ensure each cell contains only atomic (indivisible) values.
2NF: Ensure that all non-key attributes are fully dependent on the primary key.
3NF: Ensure that no non-key attribute depends on another non-key attribute.

5. Denormalization:
Denormalization is the opposite of normalization. It involves combining related tables to improve query performance, evenif it introduces some redundancy. Denormalization is often used in data warehouse schema designs to optimize read-heavy operations.
6. Relationships:
One-to-One: Each record in Table A relates to one record in Table B (e.g., each person has one passport).
One-to-Many: A single record in Table A can relate to multiple records in Table B (e.g., a customer can place multiple orders).
Many-to-Many: Multiple records in Table A can relate to multiple records in Table B (e.g., a student can enroll in multiple courses, and a course can have multiple students). A junction table is often used to implement this relationship.
7. Indexes:
Indexes are used to speed up the retrieval of records from a database. They create a data structure that makes it quicker to find rows in a table based on the values of one or more columns.

This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

Unit 2

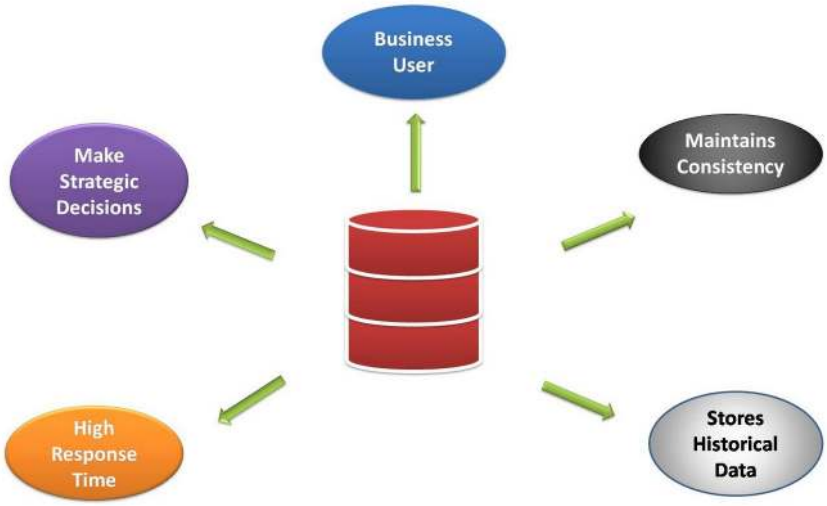
Data warehouse Process and technology(DWM)

Warehousing Strategy

A warehouse strategy is a plan that outlines how you will manage your warehouse for optimal efficiency. It's important to note that a warehouse plan is not the same thing as a strategy. A well-crafted warehouse strategy focuses on how well your warehouse operates within given benchmarks (e.g., storage density, fulfilment speed, inventory accuracy, etc.)

- Outline your data stack: including ETL process, data pipeline tools, and data science pipeline
- Define data lifecycle requirements
- Determine actions you expect to take on the data
- Sketch out dashboards you expect to build with the data
- Build your data analytics team
- Explore data warehouse solutions

Efficiently providing systematic, contextual data to the business intelligence tool of an organization, the data warehouses can find out more practical business strategies.



Source

- **Business User:** Business users or customers need a data warehouse to look at summarized data from the past.Since these people are coming from a non-technical background also, the data may be represented to them in an uncomplicated way.
- **Maintains consistency:** Data warehouses are programmed in such a way that they can be applied in a regular formatto all collected data from differentsources, which

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

makes it effortlessfor company decision-makers to analyse and share data insights By standardizing thedata, the risk of errorin interpretation is also reduced and improves overall accuracy.

- **Store historical data:** DataWarehouses are also used to store historical data that means, the time variabledata from the past and this input can be used for various purposes.
- **Make strategic decisions:** Data warehouses contribute to making better strategic decisions. Some businessstrategies may be depending upon the data stored within the datawarehouses.
- **High response time:** Data warehouse has got to be prepared for somewhat sudden masses and type of queries thatdemands a major degree of flexibility and fast latency.

Data Warehousing Process

There are five notable stages of data warehousing, starting with defining the business goals and how they are measured.

1.Establishing Business Objectives

Every business has objectives, and the first step in the data warehousing process is clearly determining those business objectives.example, delivering the best telecommunication services in the industry. Every department should have objectives that contribute to the larger company goal.

2.Collecting And Analysing Information

The next stage will collect and analyse the information to put into the data warehouse. Each business department will create a summary and analytical reports that include information about various business processes.

3.Developing A Data Framework

storing data in the warehouse needs to be done in an organized and efficient manner. The best way to do that is to develop a conceptual data model. A data framework will ensure that the data you hold in the warehouse is complete. The framework can take a long time to construct, but it makes the data warehousing process relatively easy.

4.Moving The Data

Every business has databases and backups, which will be the primary source of information. You will have to ensure the data is complete from the source or program it for completion before transfer.

The key is to ensure any reports generated from the data warehouse match the reports from the data source.

5.Plan Implementation

The final stage will be implementing the plan. Once you move the data and successfully track its importation into the data warehouse, you must ensure that the data fits well into the

This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

predetermined structure or framework. If it is a large project, it will include developing phases and schedules for data delivery.

Warehouse Planning and Implementation

The warehouse planning is to minimise high warehousing costs which arise, for example, from low-capacity utilisation, outdated warehouse technology or complex organisation. A further aim can be to increase the degree of automation. Implementing a data warehouse involves several critical steps, each contributing to the overall success of the data architect's project**.**

1. Requirement Gathering and Analysis

- Identifying the data sources and the type of data to be collected.
- Determining the business goals that the data warehouse should support.
- Engaging stakeholders, including business users, data engineers, and database administrators, to gather requirements

2. Designing the Data Warehouse Architecture

A well-planned **data warehouse architecture** is crucial for efficient data storage and efficient data retrieval.

- **Data Modeling**: Designing the schema, which defines how data is organized within the warehouse. Common models include star schema, snowflake schema, and data vault modeling.
- **ETL Processes**: Planning how data will be extracted from source systems, transformed to fit the warehouse schema, and loaded into the warehouse.

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

- **Data Storage Solutions**: Selecting appropriate technologies for data storage, such as relational databases or big data platforms.

3.Data Cleansing and Validation

- **Data Cleansing:** Identifying and correcting errors or inconsistencies in the data.
- **Data Validation:** Ensuring that the data meets the predefined quality criteria and business rules

4. Implementing Data Security and Compliance

Data security and compliance are paramount in any such **data warehousing** system or project.

- **Data Encryption:** Protecting data at rest and in transit through encryption.
- **Access Controls:** Implementing role-based access to restrict who can access or modify data.
- **Compliance**: Ensuring that the data warehouse complies with relevant regulations and standards.

5.Testing and Quality Assurance

- **User Acceptance Testing (UAT**): Engaging end-users to test the system and ensure it meets their needs.
- **Quality Assurance (QA):** Testing the system for data accuracy, query performance, and security.

6.Deployment and Maintenance

The data warehouse has passed all tests, it is ready for deployment. Ongoing maintenance is crucial to ensure all the data the system remains reliable and efficient.

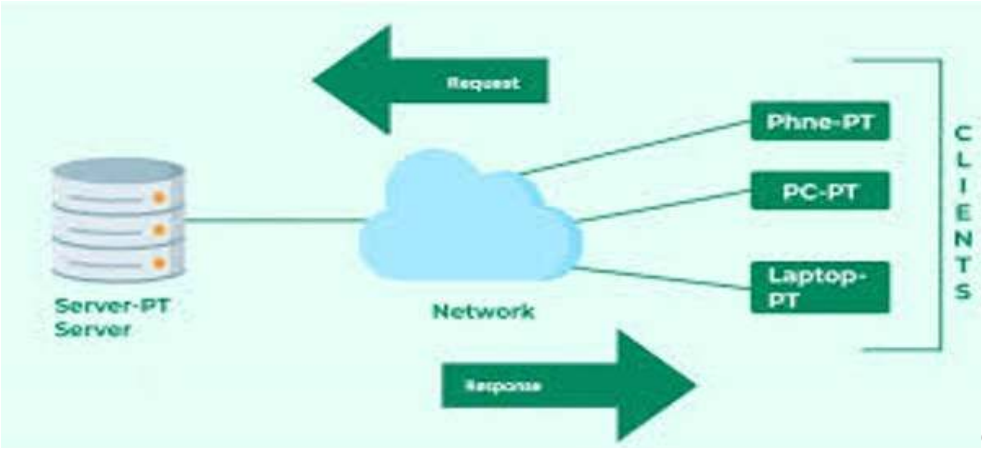
- **Monitoring:** Continuously monitoring the data warehouse for performance issues or data inconsistencies.
- **Upgrades and Scalability:** Updating the system to handle increased data volumes or new business requirements.

The client/server computing model and data warehousing

The client/server computing model plays a significant role in the architecture of data warehousing systems, providing a framework for distributing processing tasks between clients (end-users) and servers (data warehouse infrastructure). Here's how the client/server model relates to data warehousing:

This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)



Client/Server Computing Model

In the client/server computing model, computing tasks are divided between clients and servers:

1. **Client:**
   - Refers to the end-user devices, such as desktop computers, laptops, tablets, or smartphones.
   - Executes user applications and interfaces with the user.
   - Requests data and services from servers.
2. **Server:**
   - Refers to powerful computers or server clusters that provide services to clients.
   - Handles data storage, processing, and management tasks.
   - Responds to client requests by providing data or executing operations.

Role in Data Warehousing

In the context of data warehousing, the client/server model is used to facilitate access to and utilization of the data warehouse by end-users:

1. **Client-**
2. **Side Applications:**

   - Business intelligence (BI) tools, reporting applications, and analytical software installed on client devices.

   - Allows users to interact with the data warehouse, query data, generate reports, and visualize insights.

3. **Server-Side Components:**

   - Data warehouse servers and associated infrastructure.

   - Stores and manages large volumes of structured and sometimes unstructured data.

   - Executes complex data processing tasks, including ETL (Extract, Transform, Load), data aggregation, and query optimization.

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

Interaction in Data Warehousing

The client/server model facilitates the interaction between end-users and the data warehouse:

1. **Data Retrieval:**
   - Clients send queries to the data warehouse servers to retrieve specific datasets or perform analyses.
   - Servers process these queries, accessing the required data from storage, and returning the results to the clients.
2. **Data Presentation:**
   - Servers provide processed data to clients in a format suitable for analysis or visualization.
   - Clients use BI tools or reporting applications to present the data to end-users in the form of charts, graphs, tables, or dashboards.
3. **Data Manipulation:**
   - Clients can manipulate and analyse data locally using client-side applications.
   - Servers may also provide services for advanced data processing, such as machine learning or predictive analytics, depending on the architecture.

Benefits

The client/server computing model offers several benefits in the context of data warehousing:

1. **Scalability:**
   - Enables scaling of both client and server components independently based on demand.
   - Additional clients can be added without impacting server performance, and vice versa.
2. **Centralized Management:**
   - Centralizes data storage and management on servers, simplifying administration and ensuring data consistency.
3. **Resource Utilization:**
   - Distributes processing tasks between clients and servers, optimizing resource utilization and performance.

This document is available on studocu

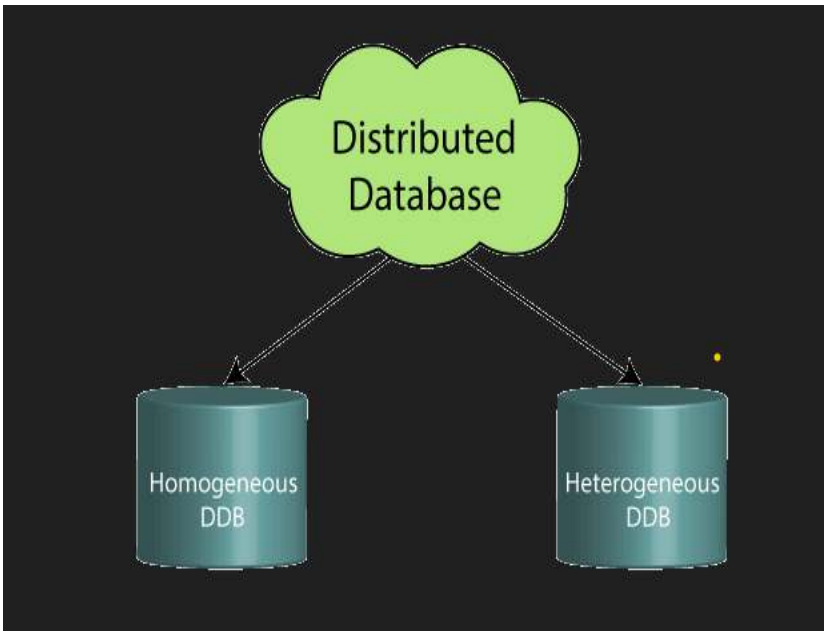
Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

4. **Flexibility:**

    - Allows users to access data warehouse resources from various client devices and locations, providing flexibility in data access and analysis.

Distributed DBMS implementations in data warehousing

A distributed database is essentially a database that is dispersed across numerous sites, i.e., on various computers or over a network of computers, and is not restricted to a single system. A distributed database system is spread across several locations with distinct physical components.



Types:

1. **Homogeneous Database:** A homogeneous database stores data uniformly across all locations. All sites utilize the same operating system, database management system, and data structures. They are therefore simple to handle.

2. **Heterogeneous Database:** With a heterogeneous distributed database, many locations may employ various software and schema, which may cause issues with queries and transactions. They could even employ separate database data models. Translations are therefore necessary for communication across various sites.

Data may be stored on several places in two ways using distributed data storage:

1. **Replication -** With this strategy, every aspect of the connection is redundantly kept at two or more locations. It is a completely redundant database if the entire database is accessible from every location. Systems preserve copies of the data as a result of replication. This has advantages since it makes more data accessible at many locations. Moreover, query requests can now be handled in parallel.
2. **Fragmentation -** In this method, the relationships are broken up into smaller pieces and each fragment is kept in the many locations where it is needed. To ensure there is no data loss, the pieces must be created in a way that allows for the reconstruction of the original relation. As fragmentation doesn't result in duplicate data, consistency is not a concern.

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

Uses for distributed databases

- The corporate management information system makes use of it.
- Multimedia apps utilize it.
- Used in hotel chains, military command systems, etc.
- The production control system also makes use of it

Characteristics of distributed databases

Distributed databases are logically connected to one another when they are part of a collection, and they frequently form a single logical database. Data is physically stored across several sites and is separately handled in distributed databases.

- Place unrelated
- Spread-out query processing
- The administration of distributed transactions
- Independent of hardware
- Network independent of operating systems
- Transparency of transactions
- DBMS unrelated

Designing Data Warehouse Schemas

Designing an effective data warehouse schema involves understanding business processes, defining dimension and fact tables, and ensuring data integrity.

Steps in Schema Design

1. **Identify Business Processes**: Determine the key processes and measures to be analysed.
2. **Define Dimension Tables**: Create tables for descriptive attributes.
3. **Create Fact Table**: Centralize measures and link to dimension tables.
4. **Normalize Data**: For snowflake schema, further break down dimension tables to reduce redundancy.
5. **Ensure Data Integrity**: Use primary and foreign keys to maintain relationships.

Benefits of Data Warehouse Schemas

- **Efficient Querying**: Well-designed schemas enhance query performance.
- **Data Integration**: Facilitates combining data from multiple sources.
- **Reduced Redundancy**: Normalization in snowflake schema lowers data duplication.
- **Enhanced <u>Data Analysis</u>**: Supports complex queries and detailed analysis.

Challenges in Data Warehouse Schemas

- **Complexity**: Designing and maintaining complex schemas can be challenging.
- **Storage Space**: Snowflake and fact constellation schemas may require more storage.
- **Query Performance**: Multiple joins in snowflake schema can impact performance.

Best Practices for Data Warehouse Schema Design

- **Understand Business Needs**: Tailor the schema to business requirements.
- **Balance Normalization**: Find a balance between reducing redundancy and maintaining performance.
- **Ensure Scalability**: Design schemas that can grow with the business.

This document is available on studocu

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)

- **Optimize Query Performance**: Use indexing and partitioning to enhance performance.

Downloaded by Geetanjali Singh (geetanjali.cs@gmail.com)